LAPORAN PRAKTIKUM BIG DATA
MINGGU 4: NOSQL DAN PEMODELAN DATA



OLEH:

ALFARIS AULIA RAHMAN

2411533006

DOSEN PENGAMPU:

AFDHAL DINILHAK, S.KOM, M.KOM.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. Pendahuluan

Praktikum NoSQL dan Pemodelan Data memperlihatkan bahwa satu kumpulan perjalanan dapat disimpan dalam beberapa model dengan konsekuensi yang berbeda. Pencarian berdasarkan ID membutuhkan akses langsung, sedangkan agregasi pendapatan, keterhubungan zona, pencarian lokasi, dan audit perubahan memerlukan bentuk penyimpanan yang sesuai dengan pertanyaannya.

Pada notebook praktikum, pencarian 200 ID melalui SQLite berindeks membutuhkan 0,0054 detik, dibanding 9,6899 detik tanpa indeks. Agregasi seluruh satu juta baris melalui DuckDB membutuhkan 0,0742 detik, sedangkan SQLite 0,6182 detik. Delta Lake menunjukkan pembacaan versi lama, penolakan skema asing, serta pengurangan 21 file aktif menjadi satu file setelah compaction. Hasil tersebut menjadi dasar rancangan penyimpanan untuk aplikasi taksi.

Data perjalanan yang digunakan adalah data sintetis dari generator K-1b pada modul dengan seed 42. Pembacaan hasil P3 pada kedua notebook menghasilkan ArrowInvalid dengan pesan Cannot yet unify dictionaries with nulls, sehingga jalur cadangan dijalankan. Satu juta baris tiruan tetap memungkinkan perbandingan model, tetapi distribusi wilayah dan arusnya tidak mewakili pola taksi New York yang sebenarnya.

B. Tujuan

1. Membedakan model key-value, document, columnar, graph, dan spatial beserta pola query yang sesuai.
2. Menyimpan data yang sama pada beberapa model dan membandingkan waktu serta kesamaan hasil query.
3. Menjalankan SQL langsung pada Parquet berpartisi dengan DuckDB serta membandingkannya dengan SQLite.
4. Menggunakan graph untuk keterhubungan zona dan STRtree untuk pencarian lokasi dengan pembanding yang jelas.
5. Mengoperasikan versi, time travel, schema enforcement, compaction, dan restore pada Delta Lake.
6. Memilih kombinasi penyimpanan untuk aplikasi taksi dengan bukti benchmark serta menjelaskan kelemahan dan konsistensi salinannya.

C. Dasar Teori dan Dataset

1. Model data dan pola akses

   NoSQL sering dimaknai sebagai Not Only SQL karena menyediakan pilihan selain tabel relasional. Bentuk data dipilih berdasarkan pola akses. Key-value melayani pengambilan lewat kunci; document menyatukan objek bersarang dan field opsional; columnar memudahkan agregasi kolom; graph menyatakan hubungan node dan edge; spatial mengindeks posisi geometri. SQLite digunakan sebagai pembanding relasional, sedangkan Delta Lake menambahkan log transaksi di atas penyimpanan Parquet.

   Tabel 1. Model dan alat yang digunakan

Model	Tool praktikum	Pola akses
Key-value	dict Python dan JSON	Satu perjalanan berdasarkan ID
Document	TinyDB MemoryStorage	Dokumen perjalanan dengan field opsional
Columnar	DuckDB dan Parquet	Agregasi banyak baris dan kolom terpilih
Graph	NetworkX DiGraph	Keterhubungan dan arus antar zona
Spatial	Shapely STRtree	Kandidat titik pada wilayah pencarian

   Tool pada praktikum berjalan pada satu proses atau mesin. Benchmark ini tidak menguji sharding, replikasi, network partition, atau perbandingan produk seperti Redis, MongoDB, dan Neo4j. Klaim waktu hanya berlaku pada query, populasi, dan run yang dinyatakan.

2. Indeks dan penyimpanan kolumnar

   Indeks pada kolom ID menyediakan struktur pencarian sehingga SQLite tidak perlu memeriksa semua baris untuk setiap ID. Dict juga mengatur nilai berdasarkan kunci, tetapi nilai JSON harus dibuka satu per satu untuk agregasi. Parquet memungkinkan column pruning, yaitu membaca kolom yang diperlukan saja. Folder partisi borough_naik memungkinkan partition pruning ketika query membatasi wilayah. Kecepatan DuckDB turut dipengaruhi pemrosesan vektor dan paralelisme engine, sehingga tidak seluruh selisih waktu berasal dari format kolom.

3. Dokumen graph dan spasial

   Embedding menyimpan data terkait di dalam dokumen yang sama agar dapat dibaca bersama. Referencing menyimpan ID menuju objek terpisah untuk mengurangi duplikasi. Graph berarah menyimpan zona sebagai node dan arus asal-tujuan sebagai edge berbobot jumlah perjalanan. STRtree mengelompokkan bounding box geometri

untuk menyaring kandidat, kemudian predicate memeriksa hubungan geometri yang tepat. Titik spasial K-6 dibuat secara sintetis dan koordinat derajat tidak langsung berarti jarak dalam meter.

4. Delta Lake dan riwayat perubahan

   Delta Lake menyimpan metadata dan aksi add/remove pada folder _delta_log. Snapshot suatu versi dibentuk dari file aktif yang dicatat oleh log sampai komit tersebut. Update dan compaction menulis file baru tanpa langsung menghapus file historis. Time travel membaca keadaan lama, sedangkan restore menjadikan keadaan versi lama sebagai keadaan terbaru melalui komit baru. VACUUM menghapus file historis yang memenuhi kebijakan retensi; praktikum hanya menjalankan dry run.

5. Data dan lingkungan eksekusi

   Generator K-1b menghasilkan 1.000.000 baris dengan 15 kolom untuk perjalanan Januari 2023. Sampel K-2 berisi 300.000 baris dan ditambahkan id_perjalanan; TinyDB mengambil 20.000 baris pertama sampel, kemudian menambah satu dokumen cuaca. K-7 mengambil sampel 200.000 baris. K-6 secara terpisah membuat 200.000 titik jemput dan 1.000 kotak pencarian dengan seed 42.

Tabel 2. Pemetaan kolom utama

Kolom modul	Kolom asal	Makna
waktu_jemput	tpep_pickup_datetime	Waktu penjemputan
zona_jemput	PULocationID	ID zona asal
zona_tujuan	DOLocationID	ID zona tujuan
wilayah_jemput	borough_naik	Label wilayah penjemputan
jarak_mil	trip_distance	Jarak dalam mil
total_bayar	total_amount	Total pembayaran USD
tip	tip_amount	Tip USD
metode_bayar	nama_pembayaran	Label metode pembayaran

Tabel 3. Versi pustaka pada kedua notebook

Pustaka	Versi output Colab
pandas	2.2.3
NumPy	2.1.3

PyArrow	23.0.1
DuckDB	1.3.2
NetworkX	3.6.1
Shapely	2.1.2
TinyDB	4.9.0
deltalake	1.6.6
reportlab	5.0.1

D. Langkah Pengerjaan Praktikum

1. Menyiapkan Lingkungan dan data contoh

   Pustaka yang dibutuhkan diperiksa dan Google Drive di-mount. Fungsi ukur menggunakan time.perf_counter, mencatat durasi empat desimal, dan menyimpan waktu presisi untuk latihan mikrodetik. Setelah pembacaan P3 gagal, bangkitkan_trips_sintetis membentuk dataset baru sesuai K-1b. Nama kolom diganti ke nama modul, wilayah diubah menjadi teks, dan sampel dibentuk dengan random_state 42.

   Output praktikum menunjukkan 1.000.000 baris dan 15 kolom, dengan waktu baca 0,9475 detik. Data sintetis dibuat pada runtime, bukan hanya membaca folder cadangan yang tersisa dari sesi sebelumnya.

2. Tabel biasa dan key value

```
KOLOM = ["id_perjalanan", "waktu_jemput", "zona_jemput", "zona_tujuan",
         "wilayah_jemput", "jarak_mil", "total_bayar", "metode_bayar"]
data = sample[KOLOM].copy()
# Model 1 - relasional: tabel SQLite
if os.path.exists(PATH_SQLITE):
    os.remove(PATH_SQLITE)
con = sqlite3.connect(PATH_SQLITE)
ukur("relasional: muat tabel", lambda: data.to_sql("perjalanan", con, index=False))
# Model 2 - key-value: kamus (dict) dengan kunci "perjalanan:<id>", nilainya teks JSON
def bangun_kv():
    return {f"perjalanan:{r['id_perjalanan']}": json.dumps(r) for r in data.to_dict("records")}
kv = ukur("key-value: bangun dict", bangun_kv)
print("jumlah key:", len(kv))
# Pertanyaan A: ambil SATU perjalanan berdasarkan id (200 id acak)
random.seed(42)
id_uji = random.sample(range(1, len(data) + 1), 200)
def cari_sql():
    cur = con.cursor()
    return [cur.execute("SELECT * FROM perjalanan WHERE id_perjalanan = ?", (i,)).fetchone() for i in id_uji]
def cari_kv():
    return [json.loads(kv[f"perjalanan:{i}"]) for i in id_uji]
r1 = ukur("cari 200 perjalanan - SQL tanpa indeks", cari_sql)
con.execute("CREATE INDEX idx_id ON perjalanan(id_perjalanan)") # buat indeks
r2 = ukur("cari 200 perjalanan - SQL dengan indeks", cari_sql)
r3 = ukur("cari 200 perjalanan - key-value", cari_kv)
print("hasil sama?", r1[0][0] == r2[0][0] == r3[0]["id_perjalanan"])
# Pertanyaan B: hitung rata-rata total bayar per wilayah (melibatkan SEMUA data)
def hitung_sql():
    return con.execute("""SELECT wilayah_jemput, COUNT(*), ROUND(AVG(total_bayar), 2)
    FROM perjalanan GROUP BY wilayah_jemput ORDER BY 2 DESC, 1""").fetchall()
def hitung_kv():
    total, jumlah = {}, {}
    for teks in kv.values():
        d = json.loads(teks) # setiap nilai harus dibuka dulu
        w = d["wilayah_jemput"]
        total[w] = total.get(w, 0) + d["total_bayar"]
        jumlah[w] = jumlah.get(w, 0) + 1
    return sorted([(w, jumlah[w], round(total[w] / jumlah[w], 2)) for w in jumlah],
                  key=lambda x: (-x[1], x[0]))
h1 = ukur("hitung per wilayah - SQL GROUP BY", hitung_sql)
```

```
h1 = ukur("hitung per wilayah - SQL GROUP BY", hitung_sql)
h2 = ukur("hitung per wilayah - key-value (buka semua nilai)", hitung_kv)
                                                    [:3])
Add code cell                                       asil sama?", h1 == h2)
⌘/Ctrl+M B
assert r1 == r2
assert [dict(zip(KOLOM,r)) for r in r2] == r3
assert h1 == h2
```

```
[relasional: muat tabel] 1.7372 s
[key-value: bangun dict] 10.5163 s
jumlah key: 300000
[cari 200 perjalanan - SQL tanpa indeks] 9.6899 s
[cari 200 perjalanan - SQL dengan indeks] 0.0054 s
[cari 200 perjalanan - key-value] 0.0018 s
hasil sama? True
[hitung per wilayah - SQL GROUP BY] 0.4039 s
[hitung per wilayah - key-value (buka semua nilai)] 2.3057 s
[('Manhattan', 116930, 18.26), ('Queens', 81377, 18.29), ('Brooklyn', 62038, 18.24)]
hasil sama? True
```

Delapan kolom sampel disimpan pada tabel SQLite perjalanan dan dict dengan kunci perjalanan:<id> serta nilai JSON. Sebanyak 200 ID acak diambil untuk pencarian tanpa indeks, dengan indeks, dan key-value. Agregasi rata-rata total_bayar per wilayah dijalankan pada SQL serta loop atas seluruh nilai JSON. Notebook membandingkan seluruh record pencarian dan hasil agregasi melalui assertion.

Tabel 4. Hasil K-2 pada notebook praktikum

Percobaan pada 300.000 baris	Waktu detik
Muat tabel SQLite	1,7372
Bangun dict	10,5163

Cari 200 ID tanpa indeks	9,6899
Cari 200 ID berindeks	0,0054
Cari 200 ID key-value	0,0018
Agregasi SQL	0,4039
Agregasi key-value	2,3057

3. Document dengan bentuk fleksibel

```
from tinydb import TinyDB, Query
from tinydb.storages import MemoryStorage
def ke_dokumen(r):
    dok = {
        "id_perjalanan": int(r.id_perjalanan),
        "waktu_jemput": r.waktu_jemput,
        "jemput": {"zona": int(r.zona_jemput), "wilayah": r.wilayah_jemput},
        "tujuan": {"zona": int(r.zona_tujuan)},
        "biaya": {"jarak_mil": float(r.jarak_mil),
                  "total": float(r.total_bayar),
                  "metode_bayar": r.metode_bayar},
    }
    # Tip hanya dicatat untuk pembayaran kartu -> field ini ada di sebagian dokumen saja
    if r.metode_bayar == "Kartu kredit":
        dok["biaya"]["tip"] = float(r.tip)
    return dok
db = TinyDB(storage=MemoryStorage)
tabel = db.table("perjalanan")
docs = [ke_dokumen(r) for r in sample.head(20_000).itertuples()]
tabel.insert_multiple(docs)
print("jumlah dokumen:", len(tabel))
print(json.dumps(tabel.all()[0], indent=2))
```

```
jumlah dokumen: 20000
{
  "id_perjalanan": 1,
  "waktu_jemput": "2023-01-10 17:57:00",
  "jemput": {
    "zona": 219,
    "wilayah": "Staten Island"
  },
  "tujuan": {
    "zona": 46
  },
  "biaya": {
    "jarak_mil": 2.0,
    "total": 17.99,
    "metode_bayar": "Kartu kredit",
    "tip": 4.81
  }
}
```

Setiap perjalanan dibentuk menjadi JSON bersarang dengan objek jemput, tujuan, dan biaya. Field biaya.tip hanya ditambahkan untuk pembayaran Kartu kredit. TinyDB menggunakan MemoryStorage. Dokumen tambahan ber-ID 999999 membawa field cuaca tanpa perubahan skema tabel.

```
Q = Query()
print("jemput di Manhattan & total > 50 :", len(tabel.search((Q.jemput.wilayah == "Manhattan") &
                                                             (Q.biaya.total > 50))))
print("dokumen yang punya field tip :", len(tabel.search(Q.biaya.tip.exists())))
print("dokumen yang TIDAK punya tip :", len(tabel.search(~Q.biaya.tip.exists())))
# Menambah field baru ("cuaca") pada satu dokumen – tanpa mengubah struktur apa pun
tabel.insert({"id_perjalanan": 999_999, "waktu_jemput": "2023-01-15 08:00:00",
              "jemput": {"zona": 161, "wilayah": "Manhattan"}, "tujuan": {"zona": 236},
              "biaya": {"jarak_mil": 1.2, "total": 14.5, "metode_bayar": "Kartu kredit", "tip": 3.0},
              "cuaca": {"hujan": True, "suhu_c": -2.0}})
print("dokumen yang punya field cuaca :", len(tabel.search(Q.cuaca.exists())))
print("total dokumen sekarang :", len(tabel))
```

```
jemput di Manhattan & total > 50 : 9
dokumen yang punya field tip : 13979
dokumen yang TIDAK punya tip : 6021
dokumen yang punya field cuaca : 1
total dokumen sekarang : 20001
```

Pada 20.000 dokumen awal, 13.979 mempunyai field tip dan 6.021 tidak. Query jemput Manhattan dengan total lebih dari 50 memperoleh sembilan dokumen. Setelah dokumen cuaca ditambahkan, jumlah menjadi 20.001, dokumen dengan tip menjadi 13.980, dan satu dokumen mempunyai field cuaca. Contoh nilai wilayah di atas berasal dari generator sintetis.

4. SQL langsung pada Parquet berpartisi

```
import duckdb
GLOB = f"{TRIPS_BERSIH}/*/*.parquet"
con_d = duckdb.connect()
# VIEW = "jendela" bernama yang membaca Parquet langsung (tidak menyalin data).
# Di sinilah nama kolom asli TLC diberi nama yang mudah dibaca.
con_d.sql(f"""
CREATE VIEW perjalanan AS
SELECT tpep_pickup_datetime AS waktu_jemput,
       PULocationID AS zona_jemput,
       DOLocationID AS zona_tujuan,
       borough_naik AS wilayah_jemput,
       trip_distance AS jarak_mil,
       total_amount AS total_bayar,
       nama_pembayaran AS metode_bayar
FROM read_parquet('{GLOB}', hive_partitioning=true)""")
SQL = """SELECT wilayah_jemput, COUNT(*) AS jumlah, ROUND(AVG(total_bayar), 2) AS rata_bayar
FROM perjalanan GROUP BY wilayah_jemput ORDER BY jumlah DESC, wilayah_jemput"""
hasil_d = ukur("DuckDB - hitung per wilayah (langsung dari Parquet)", lambda: con_d.sql(SQL).df())
print(hasil_d)
# Pembanding: SQLite (menyimpan per baris) pada data yang sama
KOLOM_S = ["wilayah_jemput", "total_bayar", "jarak_mil", "zona_jemput", "zona_tujuan"]
if os.path.exists(PATH_BARIS):
    os.remove(PATH_BARIS)
con_s = sqlite3.connect(PATH_BARIS)
ukur("SQLite - muat seluruh baris ke tabel", lambda: df[KOLOM_S].to_sql("perjalanan", con_s, index=False))
hasil_s = ukur("SQLite - hitung per wilayah", lambda: con_s.execute(
    """SELECT wilayah_jemput, COUNT(*), ROUND(AVG(total_bayar), 2)
    FROM perjalanan GROUP BY wilayah_jemput ORDER BY 2 DESC, 1""").fetchall())
print("hasil sama?", [tuple(r) for r in hasil_d.itertuples(index=False)] == [tuple(r) for r in hasil_s])
assert [tuple(r) for r in hasil_d.itertuples(index=False)] == [tuple(r) for r in hasil_s]
assert int(hasil_d["jumlah"].sum()) == len(df)
```

```
[DuckDB - hitung per wilayah (langsung dari Parquet)] 0.0742 s
  wilayah_jemput  jumlah  rata_bayar
0      Manhattan  391107       18.26
1         Queens  270929       18.27
2       Brooklyn  205487       18.29
3  Staten Island   67041       18.28
4          Bronx   60912       18.21
5            EWR    4524       18.31
[SQLite - muat seluruh baris ke tabel] 2.5862 s
[SQLite - hitung per wilayah] 0.6182 s
hasil sama? True
```

```
# Filter pada kolom partisi: DuckDB hanya membuka folder wilayah yang diminta (partition pruning, P3)
SQL2 = """SELECT COUNT(*), ROUND(AVG(total_bayar), 2)
FROM perjalanan WHERE wilayah_jemput = 'Staten Island'"""
print(ukur("DuckDB - hanya wilayah Staten Island", lambda: con_d.sql(SQL2).fetchall()))
ukuran_pq = sum(os.path.getsize(os.path.join(r, f)) for r, _, fs in os.walk(TRIPS_BERSIH) for f in fs) / 1024**2
print(f"Parquet ({df.shape[1]} kolom): {ukuran_pq:.1f} MB | SQLite ({len(KOLOM_S)} kolom): {os.path.getsize(PATH_BARIS) / 10
```

```
[DuckDB - hanya wilayah Staten Island] 0.0136 s
[(67041, 18.28)]
Parquet (15 kolom): 32.2 MB | SQLite (5 kolom): 37.6 MB
```

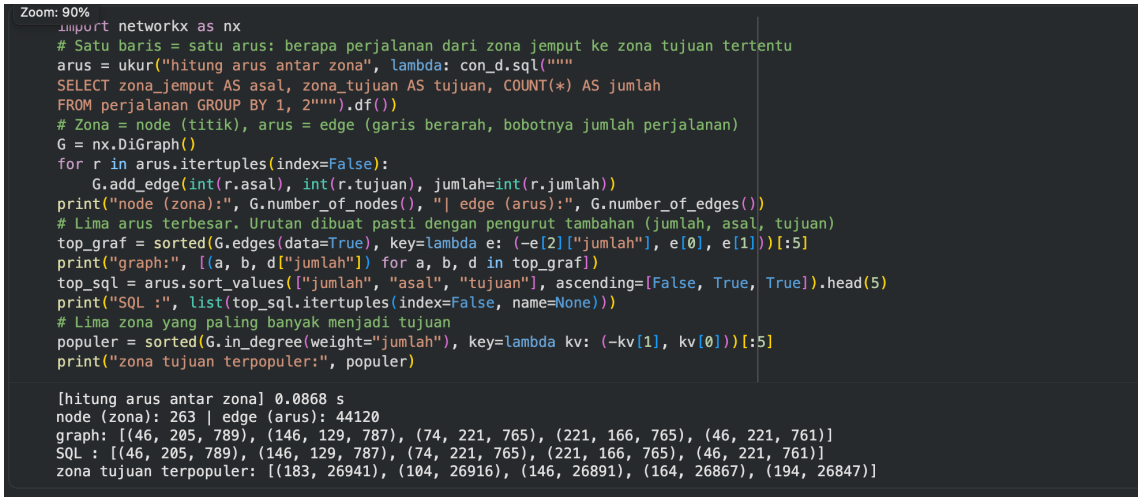
DuckDB membuat view perjalanan langsung dari Parquet dengan hive_partitioning=true. Query menghitung jumlah dan rata-rata pembayaran per wilayah, kemudian dibandingkan dengan SQLite yang memuat seluruh satu juta baris pada lima kolom. Filter Staten Island digunakan untuk memperlihatkan pembacaan satu partisi.

Tabel 5. Agregasi K-4 yang sama pada DuckDB dan SQLite

Wilayah	Jumlah	Rata bayar USD
Manhattan	391.107	18,26
Queens	270.929	18,27
Brooklyn	205.487	18,29
Staten Island	67.041	18,28
Bronx	60.912	18,21
EWR	4.524	18,31

Query penuh DuckDB 0,0742 detik, SQLite 0,6182 detik, dan muat SQLite 2,5862 detik. Filter Staten Island 0,0136 detik menghasilkan 67.041 baris dengan rata-rata 18,28 USD. Parquet 15 kolom berukuran 32,2 MB dan SQLite lima kolom 37,6 MB; perbandingan ukuran ini mempunyai jumlah kolom berbeda.

5. Graph untuk keterhubungan zona



```python
MIN = 50 # abaikan arus yang terlalu jarang (kurang dari 50 perjalanan)
zona_awal = populer[0][0] # mulai dari zona tujuan terpopuler
# Pertanyaan: zona mana yang bisa dicapai dalam TEPAT 2 langkah dari zona_awal?
# (1) cara graph: telusuri tetangga dari tetangga
def dua_langkah_graph():
    hasil = set()
    for b, d1 in G[zona_awal].items():
        if d1["jumlah"] < MIN:
            continue
        for c, d2 in G[b].items():
            if d2["jumlah"] >= MIN and c != zona_awal:
                hasil.add(c)
    return hasil
# (2) cara SQL: harus menggabungkan tabel arus dengan dirinya sendiri (self-join)
con_d.register("arus", arus)
def dua_langkah_sql():
    return set(con_d.sql(f"""
    SELECT DISTINCT a2.tujuan
    FROM arus a1 JOIN arus a2 ON a1.tujuan = a2.asal
    WHERE a1.asal = {zona_awal} AND a1.jumlah >= {MIN}
    AND a2.jumlah >= {MIN} AND a2.tujuan <> {zona_awal}""").df()["tujuan"])
hg = ukur("2 langkah - graph (NetworkX)", dua_langkah_graph)
hs = ukur("2 langkah - SQL self-join (DuckDB)", dua_langkah_sql)
print("zona awal:", zona_awal, "| terjangkau (graph):", len(hg), "| (SQL):", len(hs), "| sama?", hg == hs)
# (3) Jumlah langkah bebas: "dalam paling banyak k langkah" - cukup ganti angka k
H = nx.DiGraph([(a, b) for a, b, d in G.edges(data=True) if d["jumlah"] >= MIN])
for k in (1, 2, 3, 4):
    n = len(nx.single_source_shortest_path_length(H, zona_awal, cutoff=k)) - 1
    print(f"dalam <= {k} langkah: {n} zona")
assert hg == hs
jangkauan = {k: len(nx.single_source_shortest_path_length(H,zona_awal,cutoff=k))-1 for k in (1,2,3,4)}
```

```
[2 langkah - graph (NetworkX)] 0.0107 s
[2 langkah - SQL self-join (DuckDB)] 0.0171 s
zona awal: 183 | terjangkau (graph): 91 | (SQL): 91 | sama? True
dalam <= 1 langkah: 83 zona
dalam <= 2 langkah: 91 zona
dalam <= 3 langkah: 91 zona
dalam <= 4 langkah: 91 zona
```

Tabel arus dibentuk dengan GROUP BY pasangan zona asal-tujuan. Setiap pasangan menjadi edge berarah dan jumlahnya menjadi bobot. Graph memiliki 263 node dan 44.120 edge. Lima arus terbesar dibandingkan dengan SQL menggunakan tie-break asal dan tujuan agar hasil yang jumlahnya sama tetap terurut konsisten.

Tabel 6. Lima arus terbesar yang cocok pada graph dan SQL

Asal	Tujuan	Jumlah perjalanan
46	205	789
146	129	787
74	221	765
221	166	765
46	221	761

Zona tujuan terpopuler adalah 183 dengan 26.941 perjalanan masuk. Penelusuran dua langkah dimulai dari zona 183, memakai ambang minimal 50 perjalanan per edge dan mengecualikan zona awal dari hasil. Graph serta SQL self-join sama-sama menemukan 91 zona. Jangkauan paling banyak satu hingga empat langkah berturut-turut 83, 91, 91, dan 91 zona.

6. Indeks Spasial

```
from shapely import STRtree, box, points
# CATATAN: data TLC 2023 hanya berisi ID zona, TANPA koordinat. Titik di bawah ini SINTETIS.
rng = np.random.default_rng(42)
N = 200_000
bujur = rng.normal(-73.97, 0.06, N).clip(-74.25, -73.70) # longitude
lintang = rng.normal(40.74, 0.05, N).clip(40.50, 40.92) # latitude
titik = points(bujur, lintang)
# 1.000 wilayah pencarian berupa kotak kecil (sekitar 1 km x 1 km)
M = 1000
cx = rng.uniform(-74.05, -73.85, M)
cy = rng.uniform(40.65, 40.85, M)
kotak = [box(x - 0.005, y - 0.005, x + 0.005, y + 0.005) for x, y in zip(cx, cy)]
# Cara 1: periksa titik satu per satu dengan loop Python (hanya 5 kotak, 1.000 akan sangat lama)
def cara_loop():
    return [sum(1 for p in titik if k.contains(p)) for k in kotak[:5]]
# Cara 2: bandingkan seluruh titik sekaligus dengan NumPy (tanpa indeks spasial), 1.000 kotak
def cara_numpy():
    hasil = []
    for x, y in zip(cx, cy):
        m = (bujur >= x - 0.005) & (bujur <= x + 0.005) & (lintang >= y - 0.005) & (lintang <= y + 0.005)
        hasil.append(int(m.sum()))
    return hasil
h_loop = ukur("cara 1 - loop Python, 5 kotak", cara_loop)
h_np = ukur("cara 2 - NumPy, 1000 kotak", cara_numpy)
# Cara 3: indeks spasial STRtree (sejenis R-tree)
pohon = ukur("bangun indeks STRtree", lambda: STRtree(titik))
h_idx = ukur("cara 3 - STRtree, 1000 kotak",
             lambda: [len(pohon.query(k, predicate="contains")) for k in kotak])
print("5 kotak pertama - loop:", h_loop, "| STRtree:", h_idx[:5])
print("hasil NumPy dan STRtree sama?", h_np == h_idx, "| loop dan STRtree sama (5 pertama)?", h_loop == h_idx[:5])
print("total titik ditemukan (1000 kotak):", sum(h_idx))
assert h_np == h_idx and h_loop == h_idx[:5]


[cara 1 - loop Python, 5 kotak] 13.3685 s
[cara 2 - NumPy, 1000 kotak] 0.5398 s
[bangun indeks STRtree] 0.0686 s
[cara 3 - STRtree, 1000 kotak] 0.2356 s
5 kotak pertama - loop: [1012, 84, 228, 748, 509] | STRtree: [1012, 84, 228, 748, 509]
hasil NumPy dan STRtree sama? True | loop dan STRtree sama (5 pertama)? True
total titik ditemukan (1000 kotak): 417545
```

Shapely membentuk 200.000 titik sintetis dari bujur dan lintang di sekitar New York. Seribu kotak berukuran sisi 0,01 derajat dicari dengan tiga cara. Loop Python hanya dijalankan pada lima kotak, sedangkan NumPy dan STRtree pada seluruh 1.000 kotak. Predicate contains memeriksa titik di bagian dalam kotak.

Tabel 7. Pengukuran K-6 pada titik sintetis

Cara atau tahap	Kotak	Waktu detik
Loop Python	5	13,3685
NumPy	1.000	0,5398
Bangun STRtree	-	0,0686
Query STRtree	1.000	0,2356

Lima kotak pertama memberi jumlah 1.012, 84, 228, 748, dan 509 pada loop serta indeks. Seluruh hitungan NumPy dan STRtree cocok. Total temuan pada 1.000 kotak adalah 417.545; ini jumlah temuan per kotak, bukan titik unik, karena satu titik dapat muncul pada beberapa kotak yang beririsan.

7. Versi time travel dan schema enforcement

```
from deltalake import DeltaTable, write_deltalake
KOLOM_DELTA = ["waktu_jemput", "zona_jemput", "zona_tujuan",
               "wilayah_jemput", "jarak_mil", "total_bayar", "metode_bayar"]
data_delta = df[KOLOM_DELTA].sample(n=min(200_000, len(df)),
                                    random_state=42).reset_index(drop=True)
# Delta dikerjakan pada runtime lokal dan diarsipkan setelah seluruh langkah selesai.
PATH_DELTA = str(WORKDIR / "trips_delta")
if os.path.exists(PATH_DELTA):
    shutil.rmtree(PATH_DELTA)
# Versi 0: menulis data awal (100.000 baris)
write_deltalake(PATH_DELTA, data_delta.iloc[:100_000], mode="overwrite")
dt = DeltaTable(PATH_DELTA)
print("versi:", dt.version(), "| baris:", len(dt.to_pandas()))
# Versi 1: menambah 50.000 baris
write_deltalake(PATH_DELTA, data_delta.iloc[100_000:150_000], mode="append")
dt = DeltaTable(PATH_DELTA)
print("versi:", dt.version(), "| baris:", len(dt.to_pandas()))
# Versi 2: koreksi data. Anggap tarif Tunai di wilayah Bronx perlu ditambah 1.0
sekarang = data_delta.iloc[:150_000]
n_koreksi = int(((sekarang["wilayah_jemput"] == "Bronx") & (sekarang["metode_bayar"] == "Tunai")).sum())
print("baris yang akan dikoreksi:", n_koreksi)
dt.update(predicate="wilayah_jemput = 'Bronx' AND metode_bayar = 'Tunai'",
          updates={"total_bayar": "total_bayar + 1.0"})
print("versi:", DeltaTable(PATH_DELTA).version())
```

```
versi: 0 | baris: 100000
versi: 1 | baris: 150000
baris yang akan dikoreksi: 2517
versi: 2
```

```
# TIME TRAVEL: baca tabel sebagaimana keadaannya pada versi tertentu
v0 = DeltaTable(PATH_DELTA, version=0).to_pandas()
v1 = DeltaTable(PATH_DELTA, version=1).to_pandas()
terbaru = DeltaTable(PATH_DELTA).to_pandas()
print("baris v0:", len(v0), "| v1:", len(v1), "| terbaru:", len(terbaru))
selisih = terbaru["total_bayar"].sum() - v1["total_bayar"].sum()
print("selisih total_bayar (terbaru - v1):", round(selisih, 2), "| jumlah baris dikoreksi:", n_koreksi)
# Riwayat perubahan tabel
for h in DeltaTable(PATH_DELTA).history():
    print(h.get("version"), h.get("operation"))
# Log transaksi: satu file JSON per versi
print(sorted(os.listdir(f"{PATH_DELTA}/_delta_log")))
# PENEGAKAN SKEMA: menambah data dengan kolom asing harus DITOLAK, tabel tidak berubah
tambahan = data_delta.iloc[150_000:150_010].copy()
tambahan["kolom_asing"] = 1
try:
    write_deltalake(PATH_DELTA, tambahan, mode="append")
    print("append kolom asing: BERHASIL (tidak diharapkan)")
except Exception as e:
    print("append kolom asing DITOLAK:", type(e).__name__)
print("versi setelah percobaan gagal:", DeltaTable(PATH_DELTA).version())
assert len(v0)==100_000 and len(v1)==150_000 and len(terbaru)==150_000
assert np.isclose(selisih,n_koreksi)
assert DeltaTable(PATH_DELTA).version()==2
bukti_k7={"versi":2,"baris_v0":len(v0),"baris_v1":len(v1),
          "baris_v2":len(terbaru),"koreksi":n_koreksi,"selisih":float(selisih)}
```

```
baris v0: 100000 | v1: 150000 | terbaru: 150000
selisih total_bayar (terbaru - v1): 2517.0 | jumlah baris dikoreksi: 2517
2 UPDATE
1 WRITE
0 WRITE
['00000000000000000000.json', '00000000000000000001.json', '00000000000000000002.json']
append kolom asing DITOLAK: SchemaMismatchError
versi setelah percobaan gagal: 2
```

Tabel Delta awal ditulis dari 100.000 baris sampel. Append 50.000 baris menghasilkan versi 1. Pada versi 2, pembayaran Tunai di Bronx ditambah 1 USD pada 2.517 baris. Snapshot v0 dan v1 dibaca ulang, lalu penambahan kolom_asing dicoba untuk membuktikan penolakan skema.

Tabel 8. Perubahan tabel pada K-7

Versi	Operasi	Baris
0	WRITE awal	100.000

1	APPEND 50.000	150.000
2	UPDATE 2.517 baris	150.000

Selisih jumlah total_bayar antara v2 dan v1 adalah 2.517,00 USD, sama dengan 2.517 koreksi sebesar 1 USD. Append kolom_asing menghasilkan SchemaMismatchError dan versi tetap 2. Output riwayat menunjukkan UPDATE, WRITE, WRITE serta tiga file JSON log versi 0 sampai 2.

8. Small file problem dan compaction



```
dt = DeltaTable(PATH_DELTA)
file_awal_k8=len(dt.file_uris())
print("file data aktif sebelum:", len(dt.file_uris()), "| versi:", dt.version())
# 20 kali menambah data kecil (500 baris) - seperti data yang masuk sedikit demi sedikit
for i in range(20):
    potongan = data_delta.iloc[150_000 + i * 500: 150_000 + (i + 1) * 500]
    write_deltalake(PATH_DELTA, potongan, mode="append")
dt = DeltaTable(PATH_DELTA)
print("file data aktif setelah 20 append kecil:", len(dt.file_uris()), "| versi:", dt.version(),
      "| baris:", len(dt.to_pandas()))
file_append_k8=len(dt.file_uris())
baris_append_k8=len(dt.to_pandas())
t0 = time.perf_counter(); _ = DeltaTable(PATH_DELTA).to_pandas(); t_sebelum = time.perf_counter() - t0
# COMPACTION: gabungkan file kecil menjadi file yang lebih besar (dicatat sebagai versi baru)
hasil = dt.optimize.compact()
print("hasil compact:", {k: hasil[k] for k in ("numFilesAdded", "numFilesRemoved")})
dt = DeltaTable(PATH_DELTA)
t0 = time.perf_counter(); _ = DeltaTable(PATH_DELTA).to_pandas(); t_sesudah = time.perf_counter() - t0
print("file data aktif setelah compact:", len(dt.file_uris()), "| versi:", dt.version(), "| baris:", len(dt.to_pandas()))
print(f"waktu baca sebelum compact: {t_sebelum:.4f} s | sesudah: {t_sesudah:.4f} s")
# File lama TIDAK langsung terhapus (masih dibutuhkan untuk time travel). VACUUM yang menghapusnya.
print("file parquet di disk:", len(glob.glob(f"{PATH_DELTA}/*.parquet")))
kandidat = dt.vacuum(retention_hours=0, enforce_retention_duration=False, dry_run=True) # dry_run = hanya melaporkan
print("file yang akan dihapus VACUUM (dry run):", len(kandidat))
bukti_k8={"file_awal":file_awal_k8,"file_append":file_append_k8,
          "file_compact":len(dt.file_uris()),"baris_sebelum":baris_append_k8,
          "baris_sesudah":len(dt.to_pandas()),"versi":dt.version(),
          "parquet_disk":len(glob.glob(f"{PATH_DELTA}/*.parquet")),"kandidat_vacuum":len(kandidat)}
assert bukti_k8["baris_sebelum"]==bukti_k8["baris_sesudah"]==160_000
assert bukti_k8["file_compact"]==1 and bukti_k8["versi"]==23
for label,t in [("Delta - baca sebelum compact",t_sebelum),("Delta - baca sesudah compact",t_sesudah)]:
    catatan.append({"langkah":label,"detik":round(t,4)})
    waktu_tepat[label]=t

...
file data aktif sebelum: 1 | versi: 2
file data aktif setelah 20 append kecil: 21 | versi: 22 | baris: 160000
hasil compact: {'numFilesAdded': 1, 'numFilesRemoved': 21}
file data aktif setelah compact: 1 | versi: 23 | baris: 160000
waktu baca sebelum compact: 0.0790 s | sesudah: 0.0506 s
file parquet di disk: 24
file yang akan dihapus VACUUM (dry run): 23
```

Dua puluh append masing-masing 500 baris mensimulasikan data masuk sedikit demi sedikit. File aktif meningkat dari satu menjadi 21, jumlah baris menjadi 160.000, dan versi menjadi 22. Compaction menambah satu file serta menonaktifkan 21 file, sehingga versi 23 memiliki satu file aktif dengan jumlah baris tetap.

Tabel 9. File aktif dan baris pada K-8

Keadaan	Versi	File aktif	Baris
Sesudah K-7	2	1	150.000
Sesudah 20 append	22	21	160.000
Sesudah compact	23	1	160.000

Waktu baca turun dari 0,0790 menjadi 0,0506 detik. Di disk masih ada 24 file Parquet; VACUUM dry run melaporkan 23 kandidat, tanpa menghapusnya. File yang tidak aktif pada versi terbaru masih dibutuhkan untuk pembacaan versi historis.

9. Rekap dan ekspor hasil

```
def buat_rekap():
    a=detik("cari 200 perjalanan - SQL tanpa indeks")
    b=detik("cari 200 perjalanan - SQL dengan indeks")
    c=detik("cari 200 perjalanan - key-value")
    sql=detik("hitung per wilayah - SQL GROUP BY")
    kvw=detik("hitung per wilayah - key-value (buka semua nilai)")
    du=detik("DuckDB - hitung per wilayah (langsung dari Parquet)")
    sq=detik("SQLite - hitung per wilayah")
    tg=detik("2 langkah - graph (NetworkX)"); ts=detik("2 langkah - SQL self-join (DuckDB)")
    tn=detik("cara 2 - NumPy, 1000 kotak"); ti=detik("cara 3 - STRtree, 1000 kotak")
    tb=detik("bangun indeks STRtree")
    return pd.DataFrame([
      {"kebutuhan":"Ambil satu data lewat id","pilihan":"Key-value atau SQLite berindeks","pembanding":"SQLite tanpa indeks"
       "angka":f"200 ID: scan {a:.4f}s; indeks {b:.4f}s; KV {c:.4f}s",
       "interpretasi":f"Indeks menghindari scan; rasio scan/indeks {a/b:.2f}x pada sampel yang sama."},
      {"kebutuhan":"Merangkum banyak baris","pilihan":"DuckDB + Parquet","pembanding":"SQLite; key-value untuk agregasi",
       "angka":f"{len(df):,} baris: DuckDB {du:.4f}s vs SQLite {sq:.4f}s; {len(data):,} baris: SQL {sql:.4f}s vs KV {kvw:.4f
       "interpretasi":f"Dua pasangan memakai populasi masing-masing yang sama; SQLite/DuckDB {sq/du:.2f}x, KV/SQL {kvw/sql:.
      {"kebutuhan":"Field opsional/berubah","pilihan":"Document (TinyDB)","pembanding":"Kolom tabel tetap",
       "angka":f"{len(tabel):,} dokumen; cuaca {len(tabel.search(Q.cuaca.exists()))}; tanpa tip {len(tabel.search(~Q.biaya.t
       "interpretasi":"Field baru diterima tanpa migrasi; validasi aplikasi diperlukan agar salah nama/tipe tidak diam-diam
      {"kebutuhan":"Keterhubungan k langkah","pilihan":"Graph untuk traversal","pembanding":"SQL self-join",
       "angka":f"2 langkah: graph {tg:.4f}s; SQL {ts:.4f}s; hasil sama {len(hg)} zona; k=1..4 {list(jangkauan.values())}",
       "interpretasi":f"Pemenang waktu run ini: {'graph' if tg<ts else 'SQL' if ts<tg else 'setara pada presisi CSV'}; graph
      {"kebutuhan":"Lokasi dalam wilayah","pilihan":"STRtree","pembanding":"NumPy tanpa indeks",
       "angka":f"1.000 kotak: NumPy {tn:.4f}s; indeks {ti:.4f}s; bangun {tb:.4f}s",
       "interpretasi":f"Rasio NumPy/query {tn/ti:.2f}x; termasuk pembangunan {tn/(tb+ti):.2f}x; hasil sama pada titik sintet
      {"kebutuhan":"Perubahan aman dan kembali ke versi lama","pilihan":"Delta Lake","pembanding":"Folder Parquet biasa",
       "angka":f"v0/v1/v2 {bukti_k7['baris_v0']:,}/{bukti_k7['baris_v1']:,}/{bukti_k7['baris_v2']:,}; append asing ditolak;
       "interpretasi":"Log menyimpan riwayat komit; snapshot lama tetap terbaca, skema dijaga, dan compaction tidak mengubah
    ])
rekap=pd.DataFrame(catatan)
rekap.to_csv(f"{DIR_SIMPAN}/benchmark_nosql.csv",index=False)
print(rekap.to_string(index=False))
rekap_model=buat_rekap()
rekap_model.to_csv(f"{DIR_SIMPAN}/rekap_model_nosql.csv",index=False)
display(rekap_model)
```

```
                                        langkah    detik
                          baca trips_bersih (sintetis)  0.9475
                                relasional: muat tabel  1.7372
                                key-value: bangun dict 10.5163
                    cari 200 perjalanan - SQL tanpa indeks  9.6899
                   cari 200 perjalanan - SQL dengan indeks  0.0054
                          cari 200 perjalanan - key-value  0.0018
                        hitung per wilayah - SQL GROUP BY  0.4039
      hitung per wilayah - key-value (buka semua nilai)  2.3057
    DuckDB - hitung per wilayah (langsung dari Parquet)  0.0742
                    SQLite - muat seluruh baris ke tabel  2.5862
                         SQLite - hitung per wilayah  0.6182
                    DuckDB - hanya wilayah Staten Island  0.0136
                               hitung arus antar zona  0.0868
                          2 langkah - graph (NetworkX)  0.0107
                       2 langkah - SQL self-join (DuckDB)  0.0171
                            cara 1 - loop Python, 5 kotak 13.3685
                               cara 2 - NumPy, 1000 kotak  0.5398
                                    bangun indeks STRtree  0.0686
                             cara 3 - STRtree, 1000 kotak  0.2356
                             Delta - baca sebelum compact  0.0790
                             Delta - baca sesudah compact  0.0506
```

	kebutuhan	pilihan	pembanding	angka	interpretasi
0	Ambil satu data lewat id	Key-value atau SQLite berindeks	SQLite tanpa indeks	200 ID: scan 9.6899s; indeks 0.0054s; KV 0.0018s	Indeks menghindari scan; rasio scan/indeks 179...
1	Merangkum banyak baris	DuckDB + Parquet	SQLite; key-value untuk agregasi	1,000,000 baris: DuckDB 0.0742s vs SQLite 0.61...	Dua pasangan memakai populasi masing-masing ya...
2	Field opsional/berubah	Document (TinyDB)	Kolom tabel tetap	20,001 dokumen; cuaca 1; tanpa tip 6,021	Field baru diterima tanpa migrasi; validasi ap...
3	Keterhubungan k langkah	Graph untuk traversal	SQL self-join	2 langkah: graph 0.0107s; SQL 0.0171s; hasil s...	Pemenang waktu run ini: graph; graph dipilih k...
4	Lokasi dalam wilayah	STRtree	NumPy tanpa indeks	1.000 kotak: NumPy 0.5398s; indeks 0.2356s; ba...	Rasio NumPy/query 2.29x; termasuk pembangunan ...
5	Perubahan aman dan kembali ke versi lama	Delta Lake	Folder Parquet biasa	v0/v1/v2 100,000/150,000/150,000; append asing...	Log menyimpan riwayat komit; snapshot lama tet...

Catatan waktu dijadikan benchmark_nosql.csv dan tabel enam kebutuhan dijadikan rekap_model_nosql.csv dengan interpretasi. Sesudah latihan restore, ZIP tabel Delta dibuat agar mencakup versi 24. Notebook juga menghasilkan diagram, laporan studi kasus, dan validasi_run.json. Keluaran utama berada pada MyDrive/BigData/Praktikum4, sedangkan hasil notebook tugas berada pada subfolder Tugas.

Tabel dan print merupakan output yang tersimpan pada notebook. SQLite, data sintetis, dan tabel Delta aktif dikerjakan pada direktori runtime; tabel Delta akhir disalin ke Drive sebagai ZIP. Angka analisis berikut menggunakan run praktikum yang sama, sedangkan angka tugas dinyatakan tersendiri.

E. Analisis Hasil

1. Pencarian satu data

Pertanyaan O-1: berapa kali tabel tanpa indeks lebih lambat daripada tabel berindeks, berapa kali tabel berindeks lebih lambat daripada key-value, dan pada selisih mana perbedaan benar-benar berarti?

Tanpa indeks membutuhkan 9,6899 detik, berindeks 0,0054 detik, dan key-value 0,0018 detik untuk 200 ID. Rasio tanpa indeks terhadap berindeks adalah 1.794,43 kali, sedangkan berindeks terhadap key-value 3,00 kali. Indeks mengurangi pekerjaan dari scan seluruh tabel menjadi pencarian pada struktur indeks.

Selisih scan dan indeks mencapai 9,6845 detik, jauh lebih berarti pada run ini dibanding selisih indeks dan key-value yang hanya 3,600 milidetik untuk satu batch. Prediksi scan 500-1.500 kali lebih lambat terlampaui oleh hasil sekitar 1.794 kali. Key-value tiga kali lebih cepat daripada SQL berindeks sesuai rentang prediksi 2-4 kali, tetapi selisih absolutnya kecil.

2. Harga kesederhanaan key value

Pertanyaan O-2: berapa kali key-value lebih lambat untuk menghitung dibanding SQL GROUP BY, dan mengapa?

Pada 300.000 baris yang sama, key-value membutuhkan 2,3057 detik dan SQL 0,4039 detik, sehingga key-value sekitar 5,71 kali lebih lambat. Setiap JSON perlu di-decode untuk memperoleh wilayah dan total, lalu diakumulasi pada loop Python. SQL sudah mengetahui kolom numerik dan menjalankan agregasi pada engine. Hasil agregasi

cocok, sehingga perbandingan benar-benar untuk pertanyaan yang sama. Rasio aktual berada pada prediksi 5-10 kali.

3. Keuntungan dan risiko dokumen fleksibel

   Pertanyaan O-3: sebutkan satu keuntungan dan satu risiko dokumen ke-20.001 membawa field baru.

   Dokumen cuaca meningkatkan jumlah dari 20.000 menjadi 20.001 tanpa ALTER TABLE. Satu dokumen mempunyai field cuaca, 13.980 mempunyai tip, dan 6.021 tidak mempunyai tip. Keuntungannya adalah data opsional dapat di-embed sesuai objek tanpa memaksakan struktur identik. Risikonya, salah ejaan seperti biya atau tipe total berupa teks dapat diterima dan membuat query melewatkan data. Karena itu skema fleksibel tetap membutuhkan validasi aplikasi.

4. Penyimpanan per kolom

   Pertanyaan O-4: berapa kali DuckDB lebih cepat daripada SQLite, dan apa dua penyebabnya dengan satu penyebab di luar format kolom?

   DuckDB 0,0742 detik dan SQLite 0,6182 detik memberi rasio SQLite/DuckDB 8,33 kali pada satu juta baris. Hasilnya sama. Column pruning mengurangi kolom yang dibaca, sedangkan pemrosesan vektor dan paralelisme engine juga dapat menurunkan biaya komputasi. Faktor engine tersebut tidak hanya berasal dari format Parquet. Waktu muat SQLite 2,5862 detik dicatat terpisah dan tidak ditambahkan pada rasio query. Prediksi 5-20 kali sesuai dengan hasil sekitar 8,33 kali.

5. Graph untuk keterhubungan

   Pertanyaan O-5: jika dua langkah pada graph dan SQL sama cepatnya, apa yang membuat graph layak dipilih? Rujuk hasil k satu sampai empat.

   Graph 0,0107 detik dan SQL 0,0171 detik sama-sama menghasilkan 91 zona. Graph lebih cepat sekitar 1,60 kali pada run praktikum. Jangkauan untuk k=1, 2, 3, dan 4 adalah 83, 91, 91, dan 91 zona. Keunggulan pemodelan graph adalah perubahan cutoff k serta penelusuran tetangga/jalur yang langsung mengikuti relasi node-edge. SQL dapat memakai self-join atau recursive CTE, sehingga graph bukan satu-satunya cara menjawabnya.

   TEPAT dua langkah pada K-5 berarti terdapat lintasan dua edge, bukan bahwa jarak terpendek harus dua. Sebaliknya, query paling banyak k langkah menggunakan

panjang jalur terpendek dengan cutoff. Pada data sintetis, jangkauan yang cepat penuh tidak dijadikan kesimpulan tentang rute taksi nyata. Run tugas bahkan memberi SQL lebih cepat; hal itu menegaskan bahwa pilihan graph tidak dapat dibenarkan hanya dengan satu angka waktu.

6. Pembanding pada indeks spasial

   Pertanyaan O-6: berapa kali STRtree lebih cepat daripada loop Python dan NumPy, dan mengapa kedua rasio berbeda jauh?

Tabel 10. Rasio O-6 dengan cakupan pembanding jelas

Perbandingan	Pekerjaan sama	Rasio
Loop / query STRtree	Lima kotak; indeks sudah dibuat	6.312,91 kali
NumPy / query STRtree	1.000 kotak	2,29 kali
NumPy / bangun dan query STRtree	1.000 kotak	1,77 kali

Loop lima kotak 13,3685 detik dibandingkan dengan query STRtree lima kotak 0,002118 detik pada pengukuran tambahan O-6. Rasio sekitar 6.312,91 kali menggunakan waktu presisi internal; angka waktu yang ditampilkan sudah dibulatkan. Untuk 1.000 kotak, NumPy 0,5398 detik dibanding query STRtree 0,2356 detik menghasilkan rasio 2,29 kali, atau 1,77 kali jika pembangunan indeks 0,0686 detik dihitung.

Loop memanggil operasi geometri pada setiap objek titik melalui Python, sedangkan NumPy membandingkan seluruh array secara vektorisasi. Maka baseline memengaruhi besarnya manfaat yang terlihat. Waktu loop lima kotak tidak dibagi dengan waktu indeks 1.000 kotak untuk mengklaim percepatan. Semua hasil cocok pada titik sintetis yang sama.

7. Bukti kemampuan Delta Lake

   Pertanyaan O-7: buktikan update mempertahankan keadaan lama, penulisan gagal tidak menghasilkan komit, dan jelaskan file di disk setelah compaction.

Snapshot v0 berisi 100.000 baris dan v1/v2 150.000 baris. Total v2 meningkat 2.517,00 USD setelah 2.517 koreksi sebesar 1 USD; v1 tetap terbaca sebelum koreksi. SchemaMismatchError pada append kolom asing diikuti versi yang tetap 2, sehingga tidak muncul komit append gagal. Bukti ini berfokus pada keadaan transaksi yang

terlihat pembaca. Compaction merapikan 21 file aktif menjadi satu untuk 160.000 baris. Sebanyak 24 file tetap di disk untuk mendukung time travel.

F. Studi Kasus

1. Rancangan penyimpanan untuk lima fitur

   Aplikasi taksi memerlukan detail perjalanan, dashboard pendapatan, rekomendasi zona/rute, pencarian radius 500 meter, dan audit tarif. Rancangan berikut memakai angka notebook praktikum. Model dipilih menurut kebutuhan query; kemampuan radius dan sinkronisasi produksi merupakan rancangan yang memerlukan data serta implementasi tambahan.

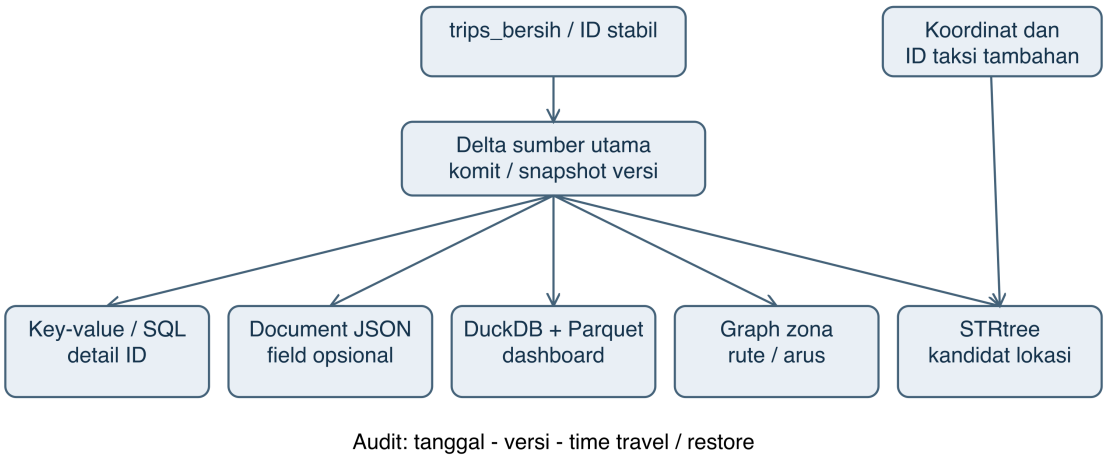
Tabel 11. Pilihan penyimpanan aplikasi taksi berdasarkan run praktikum

Fitur dan model yang dipilih	Alasan dan angka benchmark	Kelemahan
Detail perjalanan Key-value atau SQL berindeks	200 ID: KV 0,0018 s dan SQL berindeks 0,0054 s. Kunci melayani satu record; JSON menampung field opsional.	Dict berada di memori. Persistensi, ID stabil, serta sinkronisasi perlu disiapkan sebelum dipakai sebagai layanan.
Dashboard pendapatan DuckDB dan Parquet dengan Delta sebagai sumber	Agregasi satu juta baris: DuckDB 0,0742 s dibanding SQLite 0,6182 s. Query memakai kolom dan partisi yang diperlukan.	K-4 mengukur rata-rata per wilayah, bukan seluruh dashboard. Query per jam/tanggal, SUM pendapatan, dan partisi waktunya perlu dirancang.
Tujuan populer dan rute Graph berarah berbobot	Dua langkah: graph 0,0107 s dan SQL 0,0171 s, sama-sama 91 zona. Model mendukung bobot arus dan cutoff k.	Graph agregat kehilangan detail trip/waktu dan bukan jaringan jalan. Pola sintetis tidak mewakili perjalanan nyata.
Radius 500 meter STRtree dan pemeriksaan jarak	1.000 kotak: indeks 0,2356 s dibanding NumPy 0,5398 s; bangun indeks 0,0686 s.	K-6 mengukur kotak sintetis. Perlu koordinat dan ID taksi

	Kandidat lokasi disaring lebih dulu.	nyata serta CRS meter atau jarak geodesik untuk radius tepat.
Audit tarif Delta Lake	v0 berisi 100.000 baris; v1/v2 150.000. Update 2.517 baris menaikkan total 2.517 USD; append asing ditolak.	Versi harus dipetakan ke cutoff tanggal. File historis memakai ruang dan VACUUM terlalu dini memutus pembacaan versi lama.

Detail lewat ID dapat dibaca sebagai JSON tanpa mengambil relasi tambahan yang tidak diperlukan. Dashboard memerlukan agregasi berbeda, sehingga hasil K-4 mendukung pilihan engine dan format, tetapi belum membuktikan latensi semua variasi dashboard. Keputusan tersebut perlu divalidasi kembali pada beban kerja layanan sebenarnya.

2. Diagram aliran data



Gambar 1. Rancangan aliran data dari trips_bersih ke penyimpanan tiap fitur

Delta ditempatkan sebagai sumber otoritatif yang terversi. Dari sumber tersebut dibuat proyeksi detail, dokumen, tabel analitik, graph, dan indeks spasial. Koordinat dan ID taksi digambarkan sebagai sumber tambahan karena tidak tersedia sebagai data taksi nyata pada percobaan. Diagram merupakan rancangan aplikasi; pada notebook, model eksperimen dibangun langsung dari trips_bersih untuk tujuan perbandingan.

3. Data ganda dan sinkronisasi salinan

Data ganda terdapat pada JSON detail, dokumen dengan wilayah yang di-embed, agregat dashboard, edge graph, dan indeks spasial. Setiap proyeksi menyimpan ID perjalanan stabil dan source_version. Batch perubahan atau change log memperbarui proyeksi melalui upsert idempoten, kemudian checkpoint versi dipublikasikan setelah

pekerjaan selesai. Jumlah, kunci, dan agregat direkonsiliasi terhadap snapshot Delta. Jika nama wilayah berubah, dokumen yang meng-embed label lama ikut diperbarui.

Detail pembayaran dan audit membaca snapshot otoritatif, sedangkan dashboard serta rekomendasi boleh tertinggal sesuai SLA dengan menampilkan versi atau waktu pembaruan. ID yang dibuat dari indeks sampel pada praktikum perlu diganti ID yang ditetapkan dan dipersistenkan sebelum proyeksi produksi dibentuk. Mekanisme sinkronisasi ini belum diimplementasikan sebagai sistem terdistribusi di notebook.

4. Batas fitur lokasi dan audit

Pencarian dalam radius 500 meter memerlukan posisi jemput yang sesuai dengan tiap perjalanan dan identitas taksi. STRtree terlebih dahulu menyaring kandidat dari bounding box. Jarak kemudian dihitung dalam sistem koordinat bermeter atau melalui perhitungan geodesik, sehingga hasil tepat berada dalam radius. Setelah kandidat dipetakan ke ID taksi, ID yang berulang dideduplikasi. Waktu query kotak pada K-6 tidak digunakan untuk menjanjikan SLA radius 500 meter.

Permintaan mengembalikan tarif ke tanggal 1 harus menentukan cutoff waktu dan zona waktu, lalu memetakan cutoff ke versi Delta yang telah dicatat. Time travel membaca snapshot tersebut, sedangkan restore menerbitkan snapshot lama sebagai komit baru. Kebijakan audit perlu mempertahankan file historis selama periode yang dibutuhkan. Pada latihan Q-6, restore versi 1 membentuk versi 24; pada tugas dua hari, keadaan akhir hari pertama dipetakan ke versi 0.

5. Bukti pemilihan dan batas pengukuran

Tabel 12. Bukti dan makna bagi lima fitur serta data fleksibel

Kebutuhan	Bukti praktikum	Makna bagi rancangan
Akses ID	Indeks 0,0054 s dan KV 0,0018 s untuk 200 ID	Kunci atau indeks menghindari scan.
Agregasi	DuckDB 0,0742 s dan SQLite 0,6182 s	Column pruning dan engine membantu query analitik.
Field berubah	20.001 dokumen; satu field cuaca baru	Dokumen fleksibel, dengan validasi aplikasi.
Keterhubungan	Graph 0,0107 s dan SQL 0,0171 s; 91 zona	Kemudahan traversal k menjadi alasan selain waktu.

Lokasi	NumPy/query STRtree 2,29 kali	Baseline dan biaya bangun indeks harus disebut.
Audit	Versi 0, 1, 2; penolakan skema; restore 24	Riwayat komit mendukung pembacaan/koreksi yang dapat ditelusuri.

Setiap pasangan memakai populasi yang sama: 300.000 baris pada K-2 dan satu juta pada K-4. Waktu muat, bangun indeks, dan query dibedakan. Cache dan runtime memengaruhi angka. Data sintetis memperlihatkan cara kerja model, tanpa menjadi bukti perilaku taksi New York.

G. Latihan

1. Pencarian dua ribu ID

Latihan Q-1 meminta pencarian 2.000 ID acak melalui tabel berindeks dan key-value serta rata-rata waktu per pencarian dalam mikrodetik. Daftar ID dan hasil seluruh field dipakai sebagai pembanding yang sama.

Tabel 13. Hasil Q-1 pada notebook praktikum

Model	ID	Waktu detik	Mikrodetik per ID
SQLite berindeks	2.000	0,037709	18,854572
Key-value	2.000	0,012617	6,308732

Rata-rata mikrodetik diperoleh dari total detik dibagi 2.000 kemudian dikali satu juta. SQLite berindeks menghasilkan 18,854572 mikrodetik per ID dan key-value 6,308732 mikrodetik. Hasil 2.000 record cocok. Pengukuran ini adalah rata-rata batch satu proses dan tidak mencakup biaya jaringan, membangun dict, atau membuat indeks.

2. Zona tujuan dan pembayaran tunai

Latihan Q-2 menghitung dokumen dengan zona tujuan tertentu dan metode Tunai, kemudian membandingkannya dengan SQLite. Zona yang dipilih adalah 238, yaitu modus tujuan pada transaksi Tunai dalam 20.000 sampel awal.

Tabel 14. Perbedaan populasi pada Q-2

Populasi	Jumlah hasil
TinyDB 20.000 awal dan dokumen cuaca	178
SQLite seluruh 300.000 sampel	2.169
SQLite ID 1 sampai 20.000	178

Perbedaan 178 dan 2.169 berasal dari populasi, karena TinyDB tidak memuat seluruh 300.000 sampel. Setelah ID SQL dibatasi pada 1 sampai 20.000, hasil 178 cocok dengan TinyDB. Dokumen cuaca tambahan memakai Kartu kredit, sehingga tidak masuk filter Tunai. Hasil yang berbeda dari dua populasi tersebut tidak berarti sintaks query salah.

3. SUM satu dan beberapa kolom

   Latihan Q-3 membandingkan SUM satu kolom total_bayar dengan SUM empat kolom total_bayar, jarak_mil, zona_jemput, dan zona_tujuan. Kedua query menjalani warm-up lalu lima ulangan bergantian. SUM ID zona hanya berfungsi sebagai beban membaca kolom, bukan metrik bisnis.

Tabel 15. Median waktu Q-3 setelah warm up

Kolom	Ulangan	Median detik
1 kolom	5	0,022896
4 kolom	5	0,044075

   Median satu kolom 0,022896 detik dan empat kolom 0,044075 detik, dengan rasio sekitar 1,93 kali. Satu kolom lebih cepat pada run ini, sesuai dugaan column pruning mengurangi kolom yang diperlukan. Rasio tidak harus empat kali karena biaya query juga dipengaruhi cache, kompresi, encoding, dan pemrosesan engine.

4. Out degree dan tujuan berbeda

   Latihan Q-4 menghitung lima zona dengan tujuan berbeda terbanyak menggunakan out-degree graph, lalu menyilangkannya dengan COUNT(DISTINCT tujuan) pada arus. Seluruh edge digunakan tanpa ambang MIN 50. Pengurutan tambahan zona naik digunakan untuk tie-break.

Tabel 16. Lima zona dengan out degree tertinggi

Zona asal	Tujuan berbeda
30	263
46	263
52	263
55	263
74	263

Zona 30, 46, 52, 55, dan 74 masing-masing mempunyai 263 tujuan berbeda. Hasil graph dan SQL cocok. Out-degree menghitung pasangan tujuan yang berbeda, bukan jumlah seluruh perjalanan; self-loop tetap merupakan satu edge menuju zona yang sama.

5. Ukuran kotak dan manfaat indeks

   Latihan Q-5 mengubah sisi kotak menjadi 0,02 dan 0,001 derajat. Offset kiri/kanan adalah setengah sisi. Tiap ukuran memakai 1.000 kotak, tiga ulangan, dan STRtree K-6 yang telah dibuat. Batas mask NumPy memakai pertidaksamaan ketat agar sesuai dengan predicate contains.

   Tabel 17. Hasil Q-5 pada ukuran wilayah berbeda

Sisi derajat	Median NumPy s	Median STRtree s	NumPy dibagi STRtree
0,020	0,643926	1,082397	0,59
0,001	0,564294	0,013160	42,88

   Pada sisi 0,02, NumPy 0,643926 detik dan STRtree 1,082397 detik; rasio NumPy/STRtree 0,59 menunjukkan NumPy lebih cepat. Pada sisi 0,001, NumPy 0,564294 detik dan STRtree 0,013160 detik; indeks lebih cepat sekitar 42,88 kali. Kotak kecil membuat kandidat indeks jauh lebih sedikit, sedangkan NumPy tetap memeriksa semua titik untuk tiap kotak. Biaya bangun indeks tidak dihitung karena indeks dipakai ulang.

   Jumlah temuan adalah 1.666.631 untuk sisi 0,02 dan 4.149 untuk sisi 0,001. Hitungan kedua metode selalu sama. Ukuran derajat tidak dinyatakan sebagai ukuran meter yang konstan, dan total temuan tidak sama dengan jumlah titik unik jika kotak saling beririsan.

6. Membaca versi lama dan restore

   Latihan Q-6 dijalankan sesudah compaction. Sebelum restore, tabel berada pada versi 23 dengan 160.000 baris. Pembacaan version=1 tidak mengubah versi terbaru. Restore(1) lalu membentuk versi 24 dengan 150.000 baris, dan tiga operasi terakhir adalah RESTORE, OPTIMIZE, WRITE.

   Time travel hanya membaca snapshot. Restore mengubah keadaan terbaru melalui komit baru sehingga riwayat koreksi dan compaction tetap ada. Notebook memeriksa

total hasil restore sesuai snapshot v1 dan versi 23 masih berisi 160.000 baris. ZIP akhir praktikum menyimpan keadaan setelah restore beserta file historisnya.

H. Tugas Praktikum

1. Rekap Benchmark da Interpretasi

Tabel 18. Rekap R-1 dengan satu interpretasi setiap kebutuhan

Kebutuhan dan pilihan	Angka run tugas	Interpretasi
Akses ID KV atau SQL berindeks	200 ID: scan 6,8203 s; indeks 0,0048 s; KV 0,0011 s	Indeks menghindari scan; rasio scan/indeks 1.420,90 kali pada populasi yang sama.
Agregasi DuckDB dan Parquet	Satu juta: DuckDB 0,0319 s; SQLite 0,6151 s. Sampel: SQL 0,2094 s; KV 1,4247 s	SQLite/DuckDB 19,28 kali dan KV/SQL 6,80 kali memakai dua pasangan populasi yang masing-masing sama.
Field opsional Document	20.001 dokumen; cuaca satu; tanpa tip 6.021	Field baru dapat diterima tanpa migrasi, tetapi perlu validasi nama dan tipe pada aplikasi.
Keterhubungan Graph	Graph 0,0173 s; SQL 0,0055 s; sama 91 zona	SQL lebih cepat 3,15 kali pada run tugas, sedangkan graph tetap memudahkan perubahan k.
Wilayah pencarian STRtree	NumPy 0,5554 s; indeks 0,2964 s; bangun 0,0754 s	NumPy/query 1,87 kali atau 1,49 kali dengan biaya bangun.
Perubahan dan audit Delta	v0/v1/v2 100.000/150.000/150.000; kolom asing ditolak	Log menjaga snapshot versi, skema, dan jumlah baris ketika compaction merapikan file.

Run tugas mencatat 21 baris benchmark K pada R-1 dan menambah tiga waktu operasi simulasi R-3 sebelum ekspor akhir. Tabel rekap K-9 mempunyai enam kebutuhan,

masing-masing dengan interpretasi. Waktu run tugas tidak disubstitusikan ke analisis O pada notebook praktikum; misalnya graph lebih cepat pada run praktikum, sedangkan SQL lebih cepat pada run tugas.

2. Data Zona dalam Tiga Model

   R-2 memilih zona sebagai objek utama. Tabel SQL zona_ringkas menyimpan satu baris per zona dengan jumlah berangkat, tiba, dan himpunan wilayah. Dokumen zona meng-embed ringkasan serta daftar tujuan. Graph zona/arus menyimpan edge berarah berbobot. Himpunan wilayah dipertahankan agar pemilihan satu label tidak dilakukan tanpa pemeriksaan referensi.

Tabel 19. Pertanyaan mudah dan canggung pada tiga model R-2

Model	Pertanyaan mudah	Pertanyaan canggung
Tabel SQL	Lima zona dengan penjemputan terbesar melalui ORDER BY berangkat.	Zona terjangkau dalam k langkah membutuhkan tabel edge serta self-join atau recursive CTE.
Dokumen per zona	Ringkasan dan daftar tujuan satu zona diambil dalam satu dokumen.	Jalur lintas banyak zona membutuhkan beberapa dokumen dan traversal aplikasi.
Graph zona dan arus	Tujuan langsung atau zona terjangkau dalam paling banyak k langkah.	Detail perjalanan individual dan laporan per jam/tanggal tidak tersedia pada graph arus agregat.

Contoh SQL mengurutkan zona 238, 221, 129, 164, dan 183 dengan jumlah berangkat 26.996, 26.983, 26.906, 26.843, dan 26.806. Dokumen zona 183 mempunyai label wilayah Brooklyn dari generator, ringkasan berangkat 26.806 dan tiba 26.941, serta 263 tujuan; output hanya menampilkan lima tujuan pertama secara eksplisit. Graph seluruh arus dari zona 183 mencapai 262 zona lain dalam paling banyak dua langkah. Jumlah ini berbeda dari 91 pada K-5 karena contoh R-2 tidak menerapkan ambang 50 perjalanan.

Tabel ringkas memudahkan agregasi tetapi perlu diperbarui ketika trip berubah. Embedding daftar tujuan memudahkan pembacaan bersama tetapi menduplikasi ringkasan. Graph memudahkan traversal, sementara rincian transaksi dan waktu tetap

disimpan pada sumber utama. Jumlah berangkat dan tiba dijumlahkan kembali serta diperiksa sesuai jumlah seluruh perjalanan.

3. Delta Lake Dua Hari Kerja

R-3 memakai tabel terpisah dari Delta benchmark. Hari pertama menulis 100.000 baris sebagai v0. Hari kedua menambah 50.000 baris sebagai v1 lalu mengoreksi pembayaran Tunai Bronx sebesar 2 USD per baris sebagai v2. Hari kerja merupakan tahap simulasi, bukan dua tanggal komit kalender yang berbeda; tanggal perjalanan pada data tetap Januari 2023.

Tabel 20. Simulasi dan waktu operasi R-3

Hari	Versi	Operasi	Baris	Waktu detik
Hari 1	0	WRITE awal	100.000	0,0833
Hari 2	1	APPEND 50.000	150.000	0,0488
Hari 2	2	UPDATE ditambah 2 USD	150.000	0,0876

Sebanyak 2.517 baris dikoreksi, sehingga total meningkat 5.034,00 USD. Snapshot hari pertama tetap identik seluruh field pada 100.000 baris. Notebook juga membandingkan seluruh hasil hari kedua dengan gabungan input yang telah dikoreksi sesuai mask, serta memastikan pembacaan v0 tidak mengubah versi terbaru 2.

Isi log aktual dicetak untuk file 00000000000000000000.json, 00000000000000000001.json, dan 00000000000000000002.json. Log v0 berisi commit WRITE mode Overwrite, protocol, metadata skema, dan add 100.000 record. Log v1 mencatat WRITE mode Append dan add 50.000 record. Log v2 mencatat UPDATE, add file baru, dan remove file lama. Metadata protocol menggunakan minReaderVersion 3 dan minWriterVersion 7 dengan fitur timestampNtz. Log disalin ke delta_log_dua_hari dan disertakan bersama data pada ZIP skenario.

4. Laporan dan Reproducibility

R-4 menghasilkan laporan studi kasus tiga halaman dari benchmark notebook tugas, dengan lima fitur, diagram, kelemahan, dan penjelasan duplikasi/sinkronisasi. Notebook mendefinisikan import, pembacaan/generator, model, dan seluruh variabel R sendiri. Output terakhir menyatakan R-1 sampai R-4 selesai dan mencetak lokasi PDF serta folder Tugas pada Drive.

Tabel 21. Pemisahan artefak praktikum dan tugas

Berkas hasil	Praktikum atau tugas	Isi
benchmark_nosql.csv	Masing-masing folder run	Durasi dari catatan run tersebut
rekap_model_nosql.csv	Masing-masing folder run	Enam kebutuhan beserta interpretasi
trips_delta.zip	Praktikum	Delta sesudah restore versi 24
trips_delta.zip	Tugas	Delta simulasi dua hari versi 2
trips_delta_benchmark.zip	Tugas	Delta eksperimen K sampai versi 23
delta_log_dua_hari	Tugas	Isi tiga file log skenario
laporan_rancangan_P04.pdf	Masing-masing folder run	Rancangan lima fitur dan diagram
r2_tiga_model_zona.csv dan r3_timeline.csv	Tugas	Rancangan model dan urutan hari/versi

I. Pertanyaan Evaluasi

1. Mengapa NoSQL sering dibaca “Not Only SQL”? Sebutkan dua situasi ketika tabel relasional tetap pilihan yang lebih baik.
2. Apa perbedaan BASE dan ACID? Jelaskan teorema CAP dengan satu contoh data pada aplikasi taksi yang sebaiknya memilih consistency, dan satu yang boleh memilih availability.
3. Mengapa pencarian satu data pada key-value sangat cepat, tetapi menghitung sangat lambat? Apa yang dikorbankan key-value demi kesederhanaannya?
4. Jelaskan perbedaan menyatukan data (embedding) dan merujuk (referencing) pada model document, dan satu kondisi yang membuat masing-masing lebih tepat.
5. Kapan graph lebih unggul dari SQL untuk pertanyaan keterhubungan? Berikan alasan yang tidak bergantung pada kecepatan.
6. Apa yang dilakukan indeks spasial sebelum memeriksa lokasi secara tepat? Mengapa klaim “indeks 1.000 kali lebih cepat” bisa menyesatkan?

7. Apa isi folder _delta_log/ dan bagaimana ia membuat penulisan bersifat atomik dan time travel mungkin? Mengapa VACUUM dengan retention 0 jam berbahaya?

Jawab:

1. NoSQL menyediakan tambahan model sesuai pola akses, sementara SQL tetap berguna. Tabel relasional tepat untuk transaksi pemesanan/pembayaran yang membutuhkan integritas referensial dan perubahan atomik lintas tabel, serta laporan dengan hubungan terstruktur yang perlu JOIN atau query ad hoc. K-2 menunjukkan bahwa indeks membuat tabel relasional juga cepat untuk pencarian ID.
2. ACID mencakup atomicity, consistency terhadap aturan data, isolation, dan durability. BASE adalah basically available, soft state, dan eventually consistent: salinan dapat sementara berbeda kemudian konvergen. CAP membahas pilihan ketika network partition terjadi antara consistency dalam arti keadaan terbaru yang seragam dan availability dalam arti permintaan ke node yang tidak gagal tetap memperoleh jawaban. Saldo atau audit pembayaran memprioritaskan consistency; daftar perjalanan terbaru atau rekomendasi dapat tetap tersedia dengan sedikit keterlambatan. C pada ACID berbeda makna dari consistency pada CAP. Model NoSQL tidak otomatis berarti semua produknya BASE atau tidak memiliki ACID, dan praktikum satu proses tidak menguji partition jaringan.
3. Dict menggunakan hash untuk menemukan kunci tanpa scan semua nilai. Ketika agregasi berdasarkan wilayah diperlukan, penyimpanan hanya memegang teks JSON sehingga aplikasi perlu membuka seluruh nilai, mengambil field, dan menghitungnya. Kesederhanaan lookup ditukar dengan keterbatasan query isi, relasi, dan agregasi. Indeks atau ringkasan tambahan dapat dibuat tetapi perlu dijaga konsistensinya.
4. Embedding menyatukan data terkait dalam dokumen induk, seperti jemput.wilayah pada perjalanan. Pendekatan ini tepat untuk data kecil yang dibaca bersama dan jarang berubah, dengan konsekuensi atribut dapat terduplikasi. Referencing menyimpan ID ke objek lain, seperti zona_id ke koleksi zona, dan tepat bagi data yang sering diperbarui, dipakai banyak dokumen, atau relasinya besar. Pembacaan lengkap membutuhkan lookup tambahan. Field opsional tetap perlu validasi aplikasi.
5. Graph sesuai untuk jalur, tetangga, rekomendasi, dan kedalaman traversal yang berubah. Node dan edge langsung mewakili relasi, serta cutoff k mudah diubah. SQL masih bisa

melakukan traversal melalui self-join atau recursive CTE. Ringkasan edge graph dapat kehilangan detail transaksi jika hanya menyimpan bobot jumlah. Pada run tugas, SQL justru lebih cepat daripada NetworkX; pemilihan graph tetap didukung oleh kesesuaian cara bertanya.

6. STRtree menyaring kandidat melalui bounding box geometri sebelum predicate seperti contains memeriksa relasi tepat. Klaim percepatan perlu menyebut baseline, jumlah titik/kotak, ukuran wilayah, biaya bangun, dan penggunaan ulang indeks. Loop Python dan NumPy sama-sama tanpa indeks tetapi mempunyai overhead berbeda. Eksperimen kotak pada derajat dan titik sintetis tidak membuktikan fitur radius dalam meter pada posisi taksi nyata.
7. Log berisi commitInfo, protocol, metadata, add, dan remove yang membentuk daftar file aktif tiap versi. Mekanisme komit atomik pada penyimpanan yang didukung membuat pembaca melihat snapshot sebelum atau sesudah perubahan, bukan hasil setengah transaksi. Time travel membaca keadaan lama dari log dan file historis. Update/compact membuat file baru serta menonaktifkan file lama. VACUUM yang benar-benar menghapus dengan retensi nol dapat memutus time travel dan menghapus file yang masih dibaca proses lain. Praktikum hanya memakai dry_run=True sehingga tidak menghapusnya.

Github : BigData26_A_2411533006_AlfarisAuliaRahman

Google Drive : BigData